



Unidad IV. Desarrollo de Aplicaciones web dinámicas	ACTIVIDAD: DSW-A12. Aplicación web MVC con Node JS
TIPO: Actividad individual	

OBJETIVO:

El estudiante desarrollará una aplicación web dinámica utilizando Node JS, el framework Express y el patrón de diseño Modelo–Vista–Controlador (MVC), con la finalidad de consultar información de estudiantes de una base de datos MySQL y mostrarlos mediante una vista de la aplicación.

DESCRIPCIÓN:

En esta actividad, el estudiante construirá una aplicación Web para listar información de estudiantes y sus calificaciones. Para la parte de backend empleará Node JS y el paquete **Express**, el cual es un web application framework para Node.js mínimo y flexible, que proporciona un conjunto sólido de funciones para desarrollar aplicaciones web y móviles. Las siguientes son algunas de las características principales del marco Express:

- Permite configurar middlewares para responder a solicitudes HTTP.
- Define una tabla de enrutamiento que se utiliza para realizar diferentes acciones según el método HTTP y la URL.
- Permite representar dinámicamente páginas HTML basándose en pasar argumentos a plantillas.

Para el frontend empleará Bootstrap, un framework CSS de código abierto y muy popular del mundo. Bootstrap proporciona una biblioteca de componentes predefinidos (botones, menús, formularios) basados en HTML, CSS y JavaScript.

MATERIAL Y EQUIPO REQUERIDOS:

- a) Visual Studio Code con una extensión para EJS (Embedded JavaScript), puede ser EJS Language Support (por DigitalBrainstem), o cualquier otro editor de código.
 - b) Node.js instalado. Para instalar Node.js y npm puedes seguir la práctica DSW-A12a-requisitosNodeJS.
- b) Navegador web actualizado (Chrome o Firefox).
- f) MySQL instalado y funcionando localmente.
- g) Acceso a terminal o consola.
- h) Carpeta de trabajo local para crear el nuevo proyecto MVC.



PROCEDIMIENTO:

Crear la base de datos

1. Crea una carpeta llamada DSW-A12
2. Abre Visual Studio Code.
3. Selecciona **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, selecciona la carpeta DSW-A12 recién creada.
5. Crea un nuevo archivo llamado **bd.sql** para crear la base de datos de la aplicación y agrega las siguientes instrucciones al archivo.

```
DROP DATABASE IF EXISTS clases;
CREATE DATABASE clases CHARACTER SET utf8mb4;
USE clases;

CREATE TABLE materia (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE alumno (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  promedio DOUBLE NOT NULL,
  id_materia INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_materia) REFERENCES materia(id)
);

INSERT INTO materia VALUES(1, 'Arquitecturas de Red');
INSERT INTO materia VALUES(2, 'Sistemas Operativos');
INSERT INTO materia VALUES(3, 'Estructuras de Datos');
INSERT INTO materia VALUES(4, 'Bases de Datos');
INSERT INTO materia VALUES(5, 'Desarrollo de Sistemas Web');
INSERT INTO materia VALUES(6, 'Enrutamiento Avanzado');
INSERT INTO materia VALUES(7, 'Cómputo en la Nube');
INSERT INTO materia VALUES(8, 'Programación Segura');
INSERT INTO materia VALUES(9, 'Programación en Red');

INSERT INTO alumno VALUES(1, 'Sofía Martínez López', 9.2, 5);
INSERT INTO alumno VALUES(2, 'Miguel Ángel Torres Ruiz', 8.5, 6);
INSERT INTO alumno VALUES(3, 'Carolina Herrera Gutiérrez', 9.8, 4);
INSERT INTO alumno VALUES(4, 'Laura González Sánchez', 8.0, 7);
INSERT INTO alumno VALUES(5, 'Ricardo Fernández Moreno', 7.8, 6);
INSERT INTO alumno VALUES(6, 'Ana María Sánchez Martín', 9.5, 1);
```



```
INSERT INTO alumno VALUES(7, 'Juan Carlos Navarro Jiménez', 6.9, 1);
INSERT INTO alumno VALUES(8, 'Beatriz Molina Castro', 8.3, 2);
INSERT INTO alumno VALUES(9, 'Fernando Gómez Velasco', 7.6, 2);
INSERT INTO alumno VALUES(10, 'José Luis Rodríguez Pérez', 7.4, 3);
INSERT INTO alumno VALUES(11, 'Claudia Jiménez Rivas', 9.1, 3);
```

6. Dentro de MySQL, para ejecutar el script de creación de la base de datos, ejecuta la siguiente instrucción **source**:

```
mysql> source bd.sql
```

7. Verifica que la base de datos se haya creado.

```
[mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| centro_computo |
| clases |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.00 sec)
```

8. Ahora vamos a crear el usuario para conectarse desde la aplicación. En este ejemplo le colocaremos el nombre **clases_user** con password **c1@s3s**.

```
mysql> CREATE USER clases_user@localhost IDENTIFIED BY 'c1@s3s';
Query OK, 0 rows affected (0.01 sec)
```

9. Y para darle permisos en la nueva base de datos:

```
mysql> GRANT ALL ON clases.* TO clases_user@localhost;
Query OK, 0 rows affected (0.01 sec)
```

10. Para verificar que los permisos se hayan aplicado:

```
mysql> SHOW GRANTS FOR clases_user@localhost;
+-----+
| Grants for clases_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `clases_user`@`localhost` |
| GRANT ALL PRIVILEGES ON `clases`.* TO `clases_user`@`localhost` |
+-----+
```



```
+-----+  
2 rows in set (0.00 sec)
```

11. Cerrar la sesión y salir.

```
Símbolo del sistema  
mysql> exit;  
Bye  
C:\Program Files\MySQL\MySQL Server 8.0\bin>
```



Crear la aplicación MVC.

1. Asegura tener la carpeta llamada DSW-A12
2. Abre Visual Studio Code.
3. Selecciona **Archivo** > **Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, selecciona la carpeta DSW-A12 recién creada.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
6. Abre una Terminal de consola presionando **Ctrl + ñ** o desde la opción Terminal → Nueva terminal y teclea `node -v`
7. Asegúrate de tener instalado Node.js en tu equipo. En caso de que el comando no sea reconocido, instala Node.js desde *la práctica* DSW-A12a-requisitosNodeJS, adjunta a esta actividad.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● macerika@MacBook-de-Mac DSW-A12 % node -v
v25.9.0
○ macerika@MacBook-de-Mac DSW-A12 %
```

8. Utiliza el comando `npm init -y` para crear un archivo `package.json` para la aplicación.

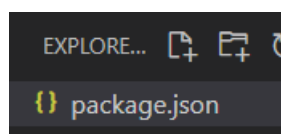
```
> npm init -y
```

9. Este comando crea el nombre, la versión de la aplicación, etc. con los parámetros predeterminados

```
● macerika@MacBook-de-Mac DSW-A12 % npm init -y
Wrote to /Users/macerika/Library/CloudStorage/OneDrive-Universi
Actividades/DSW-A12/package.json:

{
  "name": "dsw-a12",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

10. Observa que se ha creado un archivo llamado `package.json`.





11. Instala el paquete **Express** en este directorio y guárdalo en la lista de dependencias del proyecto.

```
> npm install express --save
```

```
● macerika@MacBook-de-Mac DSW-A12 % npm install express --save
added 65 packages, and audited 66 packages in 2s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

12. Observa que se ha instalado la versión. **5.2.1** de Express.

```
13. {
14.   "name": "dsw-a12",
15.   "version": "1.0.0",
16.   "description": "",
17.   "main": "index.js",
18.   "scripts": {
19.     "test": "echo \"Error: no test specified\" && exit 1"
20.   },
21.   "keywords": [],
22.   "author": "",
23.   "license": "ISC",
24.   "type": "commonjs",
25.   "dependencies": {
26.     "express": "^5.2.1"
27.   }
28. }
29.
```

30. Si aún no lo tienes, instala el paquete **ejs** (Embedded Javascript Templating) que es un template engine para Node.js.

```
npm install ejb --save
```

31. También instala el paquete **dotenv** que permite cargar variables de entorno desde un archivo con extensión **.env** hacia **process.env**. En este caso, utilizamos **dotenv** para almacenar las configuraciones de la base de datos y el puerto del servidor.

```
npm install dotenv --save
```

32. Por último instala el paquete **mysql2** que es el cliente MySQL para Node.js con enfoque en el rendimiento.



```
npm install mysql2 --save
```

33. Como dato adicional, para desinstalar un paquete, puedes utilizar `npm uninstall <package_name>`.

34. Ahora creamos el archivo `.env` en la raíz de la aplicación. Una vez creado, agrega los datos de conexión a MySQL.

```
DB_HOST=localhost
DB_DATABASE=clases
DB_USER=clases_user
DB_PASSWORD=c1@s3s
SERVER_PORT=3000
NODE_ENV=development
```

35. Ahora vamos a crear el archivo principal de Node.js. En la raíz del proyecto crea un nuevo archivo llamado `index.js` y agrega el siguiente código.

```
const express = require('express')
const dotenv = require('dotenv');
const app = express()

// Carga la configuración del archivo .env
dotenv.config();
// Establece el motor de vistas a ejs
app.set('view engine', 'ejs')

// Ruta de la URL raíz
app.get('/', (req, res) => {
  res.send("Hola mundo!")
})

// Inicia el servidor web en el puerto SERVER_PORT
app.listen(process.env.SERVER_PORT, () => {
  console.log(`Aplicación de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
})
```

*Las comillas de la línea

```
console.log(`Aplicación de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
})
```

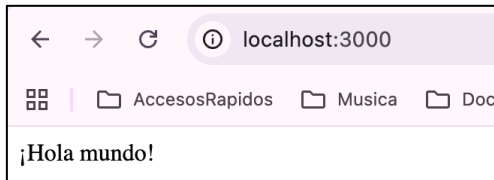
Deben ser comillas invertidas ``, para que la variable `process.env.SERVER_PORT` se interprete correctamente (interpolación de cadenas).

36. Ejecuta la aplicación con el siguiente comando desde la terminal de VS Code. Verifica que la aplicación no marque errores.

```
> node index.js
```

```
macerika@MacBook-de-Mac DSW-A12 % node index.js
◇ injected env (6) from .env // tip: % enable debugging { debug: true }
Aplicación de ejemplo escuchando en el puerto 3000
```

37. Abre un navegador web y escribe la ruta <http://localhost:3000/> para ver el resultado.



38. Para detener el servidor, solo presiona **Ctrl + C** en la consola.
39. Cada vez que se realiza un cambio en el código, es necesario repetir esta operación. Para que el servidor reinicie de manera automática, es posible utilizar el paquete **nodemon**. Este se instala como una dependencia de desarrollo con **--save-dev**, de esta forma no se copiará a producción; pues solo es un paquete empleado en este ambiente.

```
npm install nodemon --save-dev
```

40. Observa que en el archivo **package.json** se agrega en una lista de dependencias para desarrollo.

```
{ } package.json x
{ } package.json > ...
1  {
2    "name": "dsw-a12",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    > Debug
7    "scripts": {
8      "test": "echo \"Error: no test specified\" && exit 1"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "type": "commonjs",
14   "dependencies": {
15     "dotenv": "^17.4.2",
16     "ejs": "^5.0.2",
17     "express": "^5.2.1",
18     "mysql2": "^3.22.3"
19   },
20   "devDependencies": {
21     "nodemon": "^3.1.14"
22   }
23 }
```

41. En este archivo **package.json**, debajo del apartado de scripts, agrega un nombre de script para ejecutar **nodemon**, cada vez que se ejecute **node start** en la consola.

```
5    "main": "index.js",
6    > Debug
7    "scripts": {
8      "start": "nodemon index.js",
9      "test": "echo \"Error: no test specified\" && exit 1"
10   },
11   "keywords": [],
12   "author": "",
```

42. Emplea **npm start** para iniciar la aplicación. Observa la consola.



```
macerika@MacBook-de-Mac DSW-A12 % npm start
> dsw-a12@1.0.0 start
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◊ injected env (6) from .env // tip: ~ auth for age
Aplicación de ejemplo escuchando en el puerto 3000
```

43. Cada vez que se realice un cambio en el código, el servidor se reiniciará de manera automática.

44. Cambia el mensaje en el método `get`.

```
// Ruta de la URL raíz
app.get('/', (req, res) => {
  res.send("Hola mundo con nodemon!")
})
```

45. Observa como se reinicia el servidor y puedes ver los cambios de inmediato al recargar el navegador.

```
macerika@MacBook-de-Mac DSW-A12 % npm start
> dsw-a12@1.0.0 start
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◊ injected env (6) from .env // tip: % suppress logs
Aplicación de ejemplo escuchando en el puerto 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
◊ injected env (6) from .env // tip: % multiple files
Aplicación de ejemplo escuchando en el puerto 3000
```



Agregar la funcionalidad MVC

1. En la barra de herramientas superior del **Explorador de archivos**, selecciona **Nueva carpeta** y crea las siguientes carpetas:
 - **models**. Aquí almacenaremos los modelos.
 - **views**. Aquí van las vistas. Debemos crear subcarpetas con el nombre de cada controlador.
 - **controllers**. Aquí van los controladores.
 - **data**. Aquí va la capa para el acceso a los datos.
 - **routes**. Aquí va la definición de las rutas que llaman a los controladores.
 - **middlewares**. Aquí va funcionalidad extra que puede utilizar la aplicación.
2. El árbol de tus archivos debería ser similar a este.

```
> controllers
> data
> middlewares
> models
> node_modules
> routes
> views
⚙ .env
📄 bd.sql
📄 index.js
📄 package-lock.json
📄 package.json
```

3. Vamos ahora a crear las siguientes partes de la aplicación.



Agregar los Modelos.

Hay varias maneras de conectarse a MySQL. Vamos a utilizar la manera recomendada utilizando **Grupos de conexiones** (*Connection pools*) que ayudan a reducir el tiempo dedicado a conectarse al servidor MySQL al reutilizar una conexión anterior, dejándolos abiertos en lugar de cerrarlos cuando haya terminado con ellos.

Esto mejora la latencia de las consultas, ya que evita todos los gastos generales que conlleva establecer una nueva conexión

1. Dentro de la carpeta `data`, agrega un archivo llamado `datacontext.js` y escribe el siguiente código.

```
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_DATABASE,
});

module.exports = pool
```

2. La conexión ahora está contenida en la constante `pool` que podemos utilizar en los modelos.
3. Agrega un nuevo archivo en la carpeta `models` llamado `alumno.js` con el siguiente código.

```
const pool = require('../data/datacontext')

const list = async function () {
  // El pool se inicializó en data/db.js
  // La conexión se cierra automáticamente cuando el query se resuelve
  const [rows, fields] = await pool.query(
    `SELECT alumno.id, alumno.nombre, alumno.promedio, materia.nombre AS materia_nombre
    FROM materia INNER JOIN alumno ON materia.id=alumno.id_materia`);
  // Regresa las filas
  return rows;
}

module.exports = { list }
```

4. Utilizaremos este modelo para recibir los alumnos de la base de datos.



Agregar los Controladores

1. Agrega un nuevo archivo llamado `home.js` en la carpeta `controllers` con el siguiente código.

```
const alumno = require("../models/alumno");

// home/index
const index = async function (req, res, next) {
  try {
    let items = await alumno.list()
    res.render("home", {
      model: items
    });
  } catch (err) {
    next(err);
  }
};

module.exports = { index };
```

2. Ahora vamos a crear las rutas.



Agregar las rutas.

Cuando un cliente realice una petición por medio de un **endpoint** de HTTP (GET, POST, PUT, DELETE), necesitamos determinar cómo responderá el servidor estableciendo las rutas.

1. Dentro de la carpeta **routes** agrega un nuevo archivo llamado **home.js**.
2. Agrega el siguiente código.

```
const express = require('express')
const router = express.Router()
const { index } = require("../controllers/home")

// Establece las rutas donde se encuentran los controladores
router.use("/", index)

module.exports = router
```

3. En este archivo establecemos que la ruta **/home** será manejada por el controlador **/controllers/home**.
4. Vamos ahora a crear las siguientes partes de la aplicación.



Agregar las vistas.

1. Asegúrate de haber creado la carpeta llamada **views**. Ahí es donde colocaremos los templates de HTML que utiliza el *template engine* **ejs**.
2. Dentro de esta carpeta, agrega una nueva carpeta llamada **home** y dentro un archivo llamado **index.ejs**, escribe el siguiente código.

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de productos - Universidad Veracruzana</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <script defer src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</head>

<body class="mt-4">
  <header>
    <a href="https://www.uv.mx/" class="text-bg-success position-absolute top-0 end-0 btn btn-link btn-sm px-3 py-0 rounded-0
    | text-decoration-none" title="Abrir el sitio web de la Universidad Veracruzana">Universidad Veracruzana</a>
  </header>
  <div class="container">
    <h1 class="display-6 text-center">Alumnos con materias</h1>
    <h4>Lista de alumnos</h4>

    <div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-4 mt-3">
      <% locals.model.forEach(function (item) { %>
        <%- include('_partialcard', {item: item}); %>
      <% } %>
    </div>
  </div>

  <footer class="border-top footer text-muted mt-5">
    <div class="container text-end">
      Proyecto de clase MVC &copy; 2024
    </div>
  </footer>
</body>

</html>
```

3. Observa las etiquetas de **ejs**. Observa la llamada a un **componente** (**_partialcard**). Es la manera en que el motor de vistas de **ejs** permite crear **componentes** para reutilizar el código.
4. En esta misma carpeta, agrega el nuevo archivo **_partialcard.ejs** con el siguiente código.



```
<div class="col">
  <div class="card h-100">
    <div class="card-header text-end">
      Id: <span class="badge text-bg-warning"><%= item.id %></span>
    </div>
    <div class="card-body">
      <h5 class="card-title"><%= item.nombre %></h5>
      <p class="card-text"><strong>Materia:</strong> <%= item.materia_nombre %>
      </p>
      <p class="card-text text-end"><small class="text-body-secondary"><strong>Promedio:</strong>
        <%= item.promedio %></small></p>
    </div>
    <div class="card-footer text-center"><a href="#" class="btn btn-primary btn-sm">Detalles</a></div>
  </div>
</div>
```

3. Finalmente, en la carpeta llamada `middlewares` agrega un nuevo archivo llamado `errorhandler.js`. Este se encargará de mostrar un mensaje personalizado al usuario, en caso de un error no controlado.

```
const errorHandler = (err, req, res, next) => {
  console.log("Manejo de errores desde un middleware");
  const defaultmensaje = 'La aplicación esta ocupada. Inténtelo nuevamente más tarde.'

  if (process.env.NODE_ENV === 'development') {
    const statusCode = err.statusCode || 500
    const message = err.message || defaultmensaje
    res.status(statusCode).json({
      success: false,
      status: statusCode,
      message: message,
      stack: err.stack
    })
  } else {
    res.status(200).send(defaultmensaje)
  }
}

module.exports = errorHandler
```

4. El árbol de la aplicación debería ser similar al siguiente.



```

  controllers
  JS home.js
  > data
  middlewares
  JS errorhandler.js
  models
  JS alumno.js
  > node_modules
  routes
  JS home.js
  views
    home
    <> _partialcard.ejs
    <> index.ejs
  .env
  bd.sql
  JS index.js
  {} package-lock.json
  {} package.json
```

5. Ahora es momento de configurar el programa principal.



Configurar el programa principal

1. Abre el archivo `index.js` ubicado en la raíz del proyecto y elimina todo el código en él. Agrega el siguiente código:

```
const express = require('express')
const dotenv = require('dotenv')
const app = express()

// Primero carga la configuración del archivo .env
// para que este disponible en las demás llamadas
dotenv.config();
// Establece el motor de vistas a ejs
app.set('view engine', 'ejs')
```

2. Este bloque se encarga de registrar los servicios que la aplicación utilizará y construirla.
3. Ahora vamos a crear el ambiente para las rutas de la aplicación. En el mismo archivo `index.js` agregar las siguientes líneas para definir las rutas del controlador `home.js`.

```
// Rutas
const homeRouter = require('./routes/home')
app.use("/", homeRouter)
app.use('/home', homeRouter)
```

4. En este ejemplo, tenemos dos **endpoints**, la raíz `/` y la ruta `/home`.
5. A continuación, debajo agrega las configuraciones y funcionalidades de los servicios.

```
// Middleware para el manejo de errores (Debe ser el último middleware a utilizar)
const errorHandler = require('./middlewares/errorhandler')
app.use(errorHandler)

// Inicia el servidor web en el puerto SERVER_PORT
app.listen(process.env.SERVER_PORT, () => {
  console.log(`Aplicación de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
})
```

6. La última instrucción, ejecuta la aplicación.



Probar el funcionamiento

1. En la consola escribe `nmp start`. Y verifica que no marque errores.

```
o macerika@MacBook-de-Mac DSW-A12 % npm start
> dsw-a12@1.0.0 start
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting node index.js
◊ injected env (6) from .env // tip: * override existing { override:
true }
Aplicación de ejemplo escuchando en el puerto 3000
```

2. Abre una ventana de navegador web en la ruta `http://localhost:3000/`.
3. Verifica que la aplicación muestra la lista de alumnos de manera correcta.

Alumnos con materias

Lista de alumnos

Id	Nombre	Materia	Promedio
1	Sofía Martínez López	Desarrollo de Sistemas Web	9.2
2	Miguel Ángel Torres Ruiz	Enrutamiento Avanzado	8.5
3	Carolina Herrera Gutiérrez	Bases de Datos	9.8
4	Laura González Sánchez	Cómputo en la Nube	8
5	Ricardo Fernández Moreno	Enrutamiento Avanzado	7.8
6	Ana María Sánchez Martín	Arquitecturas de Red	9.5
7	Juan Carlos Navarro Jiménez	Arquitecturas de Red	6.9
8	Beatriz Molina Castro	Sistemas Operativos	8.3
9	Fernando Gómez Velasco	Sistemas Operativos	7.6
10	José Luis Rodríguez Pérez	Estructuras de Datos	7.4
11	Claudia Jiménez Rivas	Estructuras de Datos	9.1

4. Revisa y reflexiona sobre la estructura y funcionamiento de este Framework de desarrollo. La mayoría de frameworks para desarrollo web funcionan de una manera similar.
5. Felicidades, has creado tu aplicación de manera correcta.



RESULTADOS ENTREGABLES

1. Un documento .pdf con portada y **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de la consola de MySQL con el resultado del comando para listar las bases de datos de MySQL.
 - b. Coloca la captura de pantalla de tu código fuente del archivo `index.js`.
 - c. Coloca la captura de pantalla de la estructura de tus archivos y carpetas de tu proyecto en Visual Studio Code.
 - d. Coloca la captura de pantalla de la salida de tu aplicación en un navegador web.
2. El archivo .zip con la carpeta de tu proyecto.